Pandas for AI Engineering

Detailed practical guide with examples — from fundamentals to ML-ready data preparation

Goal: Be able to load, inspect, clean, transform, combine, analyze, and prepare real-world data for machine-learning and AI pipelines.

Recommended depth: practical intermediate Pandas. You do not need to master Pandas internals or every API before moving to Scikit-learn, PyTorch, and AI engineering.

1. Setup

Install and import Pandas:

```
pip install pandas

import pandas as pd
```

The standard alias **pd** is used throughout this guide.

2. Series and DataFrame

A **Series** is one-dimensional labeled data. A **DataFrame** is a two-dimensional table of rows and columns.

```
s = pd.Series([10, 20, 30])

df = pd.DataFrame({
    "name": ["A", "B", "C"],
    "age": [20, 25, 30]
})

df.shape
df.columns
df.index
df.dtypes
df.values
```

Mental model: **Series** ≈ **one column**; **DataFrame** ≈ **a table made from multiple Series**.

3. Reading and Writing Data

Most projects begin with CSV, JSON, Excel, database, or API data.

```
df = pd.read_csv("customers.csv")
df = pd.read_json("customers.json")
df = pd.read_excel("customers.xlsx")

df.to_csv("output.csv", index=False)
df.to_json("output.json")
df.to_excel("output.xlsx", index=False)
```

Useful options:

```
df = pd.read_csv(
    "customers.csv",
    usecols=["name", "age", "income"],
    na_values=["NA", "unknown"]
)
```

For very large datasets, eventually learn databases, chunked processing, Polars, Spark, or other scalable tools rather than assuming everything should fit in memory.

4. Inspecting a Dataset

Always inspect before cleaning or modeling.

```
df.head()
df.tail()
df.sample(5)

df.shape
df.info()
df.describe()

df.columns
df.dtypes

df.isna().sum()
df["city"].unique()
df["city"].nunique()
df["city"].value_counts()
```

A strong first pass is: **shape** → **columns** → **types** → **missing values** → **duplicates** → **distributions** → **categories**.

5. Selecting Data: loc and iloc

loc is label-based; **iloc** is position-based.

```
df["name"]
df[["name", "age"]]

df.loc[0]
df.loc[0:1, ["name", "age"]]

df.iloc[0]
df.iloc[0:2, 0:2]
```

Remember: **loc** → **labels**, **iloc** → **integer positions**. This becomes important after filtering, sorting, and changing indexes.

6. Filtering

```
df[df["age"] > 25]

df[df["city"] == "Delhi"]

df[
    (df["age"] > 20) &
    (df["city"] == "Delhi")
]

df[
    (df["age"] > 25) |
    (df["city"] == "Delhi")
]

df[df["city"].isin(["Delhi", "Indore"])]
```

Use **&** for element-wise AND and **|** for element-wise OR. Put each condition in parentheses.

7. Creating, Modifying, Renaming and Dropping Columns

```
df["salary"] = [30000, 40000, 50000]

df["salary_lakh"] = df["salary"] / 100000

df = df.rename(
    columns={"salary": "annual_salary"}
)

df = df.drop(columns=["salary_lakh"])
```

Prefer clear assignments over relying heavily on **inplace=True**.

8. Sorting

```
df.sort_values("age")

df.sort_values(
    "age",
    ascending=False
)

df.sort_values(
    ["city", "age"],
    ascending=[True, False]
)
```

Sorting is useful for exploration, ranking, top/bottom records, and reports.

9. Missing Values — Essential

Real datasets contain missing values. Detect them first; then choose a strategy based on what the missing value means.

```
df.isna()
df.isnull()
df.isna().sum()
df.notna()

df.dropna()

df.dropna(
    subset=["age", "income"]
)

df["age"] = df["age"].fillna(
    df["age"].median()
)

df["city"] = df["city"].fillna("Unknown")
```

For ML, avoid leakage: statistics used for imputation or scaling should be learned from training data and then applied to validation/test data.

10. Duplicates

```
df.duplicated()
df.duplicated().sum()

df = df.drop_duplicates()

df = df.drop_duplicates(
    subset=["email"]
)
```

Do not remove duplicates blindly: first determine what counts as a true duplicate in the dataset.

11. Data Types and Conversion

```
df.dtypes

df["age"] = df["age"].astype("int64")

df["income"] = pd.to_numeric(
    df["income"],
    errors="coerce"
)

df["date"] = pd.to_datetime(
    df["date"],
    errors="coerce"
)
```

errors='coerce' converts invalid values to missing values, which you can then inspect and handle.

12. String Operations

Use the **.str** accessor for vectorized string processing.

```
df["name"].str.lower()
df["name"].str.upper()
df["name"].str.strip()

df["email"].str.contains(
    "@gmail.com", na=False
)

df["email"].str.endswith(
    "@gmail.com", na=False
)

df["name"].str.replace(
    "old", "new", regex=False
)

df["email"].str.split("@")
```

These operations are common when cleaning user data, metadata, logs, documents, and API responses.

13. GroupBy and Aggregation — Essential

GroupBy follows the pattern: **group** → **aggregate** → **inspect**.

```
df = pd.DataFrame({
    "department": ["IT", "IT", "HR", "HR"],
    "salary": [50000, 60000, 40000, 45000]
})

df.groupby("department")["salary"].mean()

df.groupby("department")["salary"].agg(
    ["mean", "min", "max", "count"]
)

df.groupby("department").agg({
    "salary": ["mean", "max"]
})
```

This is useful for data profiling, analytics, validation, and feature engineering.

14. Merge / Join — Essential

Merge combines DataFrames using keys, similar to SQL JOIN.

```
users = pd.DataFrame({
    "user_id": [1, 2, 3],
    "name": ["A", "B", "C"]
})

orders = pd.DataFrame({
    "user_id": [1, 1, 2],
    "amount": [100, 200, 150]
})

result = pd.merge(
    users,
    orders,
    on="user_id",
    how="inner"
)
```

```
inner  -> matching keys only
left   -> all left rows + matches
right  -> all right rows + matches
outer  -> all keys from both
```

You can merge when key names differ:

```
pd.merge(
    users,
    orders,
    left_on="id",
    right_on="user_id",
    how="left"
)
```

After a merge, inspect row counts. Many-to-many joins can unexpectedly multiply rows.

15. Concatenation

Use **concat** to stack compatible datasets; use **merge** when combining records through keys.

```
jan = pd.DataFrame({
    "user": ["A", "B"],
    "sales": [100, 200]
})

feb = pd.DataFrame({
    "user": ["C", "D"],
    "sales": [150, 250]
})

all_sales = pd.concat(
    [jan, feb],
    ignore_index=True
)
```

concat → **stack/combine**; **merge** → **combine using keys**.

16. Pivot Tables

Pivot tables summarize data by categories.

```
pd.pivot_table(
    df,
    values="salary",
    index="department",
    aggfunc="mean"
)

pd.pivot_table(
    df,
    values="salary",
    index="department",
    columns="city",
    aggfunc="mean"
)
```

Know what **index**, **columns**, **values**, and **aggfunc** mean. You do not need to memorize every option.

17. Apply and Map

Use **map** for value-to-value mapping and **apply** for custom functions. Prefer vectorized operations when they can do the job.

```
mapping = {"M": "Male", "F": "Female"}
df["gender"] = df["gender"].map(mapping)

df["salary"] = df["salary"].apply(
    lambda x: x * 1.10
)

df["total"] = df.apply(
    lambda row: row["price"] * row["quantity"],
    axis=1
)
```

Apply is convenient, but built-in Pandas/NumPy operations are often clearer and faster.

18. Dates and Time Series

```
df["date"] = pd.to_datetime(df["date"])

df["year"] = df["date"].dt.year
df["month"] = df["date"].dt.month
df["day"] = df["date"].dt.day
df["day_name"] = df["date"].dt.day_name()

df = df.set_index("date")
monthly = df.resample("ME").sum()
```

This matters for logs, transactions, usage metrics, monitoring, trading data, and other time-based datasets.

19. JSON and Nested Data

AI engineering frequently consumes nested JSON from APIs.

```
data = [
    {
        "name": "A",
        "address": {
            "city": "Delhi",
            "pin": 110001
        }
    }
]

df = pd.json_normalize(data)
```

Nested fields become tabular columns such as **address.city** and **address.pin**.

20. Pandas + NumPy

Pandas handles labeled/tabular data; NumPy handles numerical arrays. They work together constantly.

```
import numpy as np
import pandas as pd

df = pd.DataFrame({
    "age": [20, 25, 30],
    "salary": [30000, 40000, 50000]
})

df["normalized_salary"] = (
    df["salary"] - df["salary"].mean()
) / df["salary"].std()

arr = df.to_numpy()

new_df = pd.DataFrame(arr)
```

Be comfortable switching between a DataFrame and a NumPy ndarray when a library requires a particular representation.

21. Pandas + Machine Learning

A common workflow is: **raw data** → **inspect** → **clean** → **transform** → **select X/y** → **split** → **preprocessing/model pipeline** → **train** → **evaluate**.

```
df = pd.read_csv("customers.csv")

# Inspect
print(df.shape)
print(df.info())
print(df.isna().sum())
```

```
# Clean
df = df.drop_duplicates()
df["age"] = df["age"].fillna(df["age"].median())

# Features and target
X = df[["age", "income", "score"]]
y = df["purchased"]
```

```
from sklearn.model_selection import train_test_split

X_train, X_test, y_train, y_test = train_test_split(
    X, y,
    test_size=0.2,
    random_state=42
)
```

For learned preprocessing such as imputation and scaling, fit on training data and apply to held-out data. Scikit-learn Pipelines are useful for preventing leakage.

22. Advanced Topics — Learn Later

These are useful but should not delay your AI roadmap:

Topic	When to learn
MultiIndex	Hierarchical indexes and complex grouped data
Advanced reshaping	Specialized analytical datasets
Categorical dtype	Repeated categories, semantics, memory
Memory optimization	Larger datasets
Chunked reading	Files too large for memory
Performance profiling	When Pandas becomes a bottleneck
Copy/view behavior	Avoiding unintended mutation and indexing issues
Pandas internals	Mostly unnecessary for normal AI engineering

23. Complete Practice Project

Take one real dataset and repeat this workflow until you can do it without a tutorial:

```
# 1. Load
df = pd.read_csv("dataset.csv")

# 2. Inspect
print(df.shape)
print(df.head())
print(df.info())
print(df.describe())
print(df.isna().sum())

# 3. Clean
df = df.drop_duplicates()

# 4. Types
df["date"] = pd.to_datetime(df["date"], errors="coerce")

# 5. Missing values
df["age"] = df["age"].fillna(df["age"].median())

# 6. Filter
filtered = df[df["age"] >= 18]

# 7. Transform
df["income_k"] = df["income"] / 1000

# 8. Group
summary = df.groupby("city")["income"].agg(
```

```
    ["mean", "min", "max", "count"]
)

# 9. ML-ready data
X = df[["age", "income_k"]]
y = df["target"]
```

The goal is to learn the **workflow**, not memorize isolated methods.

24. Practice Exercises

Exercise: Create a DataFrame containing name, age, city, and salary for 8 people.

Exercise: Load a CSV and report its shape, columns, types, and missing-value counts.

Exercise: Select rows where age > 25 and salary > 40,000.

Exercise: Fill missing numerical values with the median.

Exercise: Remove duplicate records based on email.

Exercise: Group employees by department and calculate mean/min/max salary.

Exercise: Merge users and orders using user_id.

Exercise: Concatenate two monthly datasets.

Exercise: Convert a date column to datetime and extract year/month.

Exercise: Use json_normalize on a nested API response.

Exercise: Prepare X and y for a classification model.

Exercise: Perform the complete load → inspect → clean → transform → analyze → ML-ready workflow.

25. AI Engineer Checklist

Must master: DataFrame/Series, read_csv/read_json, inspection, loc/iloc, filtering, column operations, missing values, duplicates, type conversion, GroupBy/agg, merge/join, concat, Pandas + NumPy, and ML-ready preprocessing.

Should know well: strings, dates, apply/map, pivot tables, JSON normalization.

Learn later: MultiIndex, advanced reshaping, memory/performance optimization, Pandas internals.

26. Recommended 10-Day Plan

Day 1: Series/DataFrame + loading + inspection.
Day 2: loc/iloc + selection.
Day 3: filtering + sorting + columns.
Day 4: missing values + duplicates + types.
Day 5: GroupBy + aggregation.
Day 6: merge/join/concat.
Day 7: strings + dates + apply/map.
Day 8: pivot + JSON + NumPy integration.
Day 9: ML preprocessing + leakage awareness.
Day 10: complete real-dataset project.

27. Where Pandas Fits in AI Engineering

Python → NumPy → Pandas → SQL → Statistics/Math → Scikit-learn → PyTorch → Deep Learning → Transformers/LLMs → Embeddings → RAG → Agents → Production AI Engineering

Pandas is a supporting tool, not the destination. Once you can reliably clean and transform unfamiliar datasets, shift more study time toward ML concepts, deep learning, LLMs, RAG, agents, evaluation, APIs, deployment, and production systems.